TASKNEST-COLLABORATIVE AI POWERED NOTEPAD

A PROJECT REPORT

Submitted by

DHANUSHREE A **(714024247023)**

DHARANIDHARAN J **(714024247025)**

KAVYA SREE A **(714024247054)**

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SRI SHAKTHI

INSTITUTE OF ENGINEERING AND TECHNOLOGY

ANNA UNIVERSITY: CHENNAI 600025

APRIL-2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled **"TASKNEST AI INTELLIGENT WORKSPACE FOR DEVELOPERS "**Is the Bonafide work of **"DHANUSHREE A(714024247023), DHARANIDHARAN J (714024247025), KAVYA SREE A (714024247054)"**, who carried out the project work under my guidance and supervision.

SIGNATURE

Mrs. S. Hemalatha

HEAD OF THE DEPARTMENT

Artificial Intelligence and

Machine Learning.

Sri Shakthi Institute of Engineering

and Technology,

Coimbatore – 641 062

SIGNATURE

Mrs. R. Kirthiga

SUPERVISOR

Artificial Intelligence and

Machine Learning.

Sri Shakthi Institute of Engineering

and Technology,

Coimbatore – 641 062

Submitted for the project work Viva Voice Examination held on ……

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGMENT

First and foremost, I would like to thank God Almighty for giving me strength Without his blessings, this achievement would not have been possible.

We express our deepest gratitude to our Chairman **Dr. S. Thangavelu** for his continuous encouragement and support throughout our course of study.

We are thankful to our Secretary **Dr. T. Dheepan** for his unwavering support during the entire course of this project work.

We are also thankful to our Joint Secretary **Mr. T. Sheelan** for his support during the entire course of this project work.

We are highly indebted to Principal **Dr. N. K. Sakthivel** for his support during the tenure of the project.

We sincerely thank **Mrs. S. Hemalatha**, Head of the Department of Artificial Intelligence and Machine Learning, and **Mrs. R. Kirthiga**, our Project Guide, for their invaluable guidance, technical support, and the facilities provided throughout this project. We also extend our gratitude to all the teachers and non-teaching staff of our department, as well as our family and friends, for their constant encouragement and support.

DHANUSHREE A **(714024247023)**

DHARANIDHARAN J **(714024247025)**

KAVYA SREE A **(714024247054)**

ABSTRACT

AN INTELLIGENT WORKSPACE FOR DEVELOPERS AND TEAMS

TaskNest is a web-based, agentic AI-powered project management platform designed to solve the fragmentation problem faced by developers and student teams managing multiple projects simultaneously. It provides a centralized, intelligent workspace where users can organize projects into dedicated folders, each containing its own isolated knowledge base, notes section, and task management system. The platform integrates Retrieval-Augmented Generation (RAG) technology with a multi-agent system orchestrated through LangGraph, enabling a context-aware AI chatbot that adapts its knowledge base dynamically based on the active project folder. Each project folder contains a Resources section where users can upload documents such as PDFs and DOCX files. These resources are automatically ingested, chunked, embedded using state-of-the-art embedding models, and stored in a project-specific ChromaDB vector collection. This ensures that the RAG brain is completely isolated per project — when a user navigates to a different folder, the chatbot's knowledge switches entirely to that folder's resources. The system's AI orchestrator, powered by Groq's llama3-70b model and managed through LangGraph's StateGraph, routes user queries to specialized agents: a Folder Agent for project management, a Resource Agent for document handling, a Notes Agent for intelligent note-taking, and a Task Agent for todo management. Users can interact with the chatbot using natural language to perform any platform operation automatically, eliminating manual navigation.

TABLE OF CONTENTS

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1

INTRODUCTION

In the rapidly evolving landscape of software development and academic project work, teams and individuals increasingly rely on a fragmented set of tools to manage their projects. Note-taking applications, document storage systems, task managers, and AI assistants all operate in isolation, forcing users to context-switch constantly and manually transfer information between platforms. This fragmentation leads to information loss, reduced productivity, and poor knowledge retention.The emergence of Large Language Models (LLMs) and Retrieval- Augmented Generation (RAG) technology has created an opportunity to fundamentally reimagine how project workspaces should function. Instead of static document repositories, a project folder can now have a dynamic, queryable knowledge base. Instead of passive note-taking, an AI agent can actively help organize and generate notes from conversations. Instead of rigid task management, an intelligent agent can create, update, and prioritize tasks through natural language commands.TaskNest – Agentic AI-Powered Project Management Platform is designed to address these challenges by providing a unified, intelligent workspace where every project folder acts as a self-contained environment. Each folder houses its own resource library, an isolated RAG-powered chatbot whose knowledge is strictly scoped to that folder's uploaded documents, a collaborative notes section, and a task management module.The platform's AI system is built on a multi-agent architecture orchestrated through LangGraph. The RAG chatbot serves as the primary interface and orchestrator, routing user intents to specialized agents — a Folder Agent, Resource Agent, Notes Agent, and Task Agent — each responsible for a distinct domain of operations.

CHAPTER 2

LITERATURE REVIEW

This chapter presents a comprehensive review of existing research and tools related to project management platforms, Retrieval-Augmented Generation systems, agentic AI architectures, and collaborative knowledge management. It identifies the key gaps that motivated the development of TaskNest.

2.1 LIMITATIONS OF EXISTING PROJECT MANAGEMENT TOOLS

Modern project management tools such as Notion, Trello, Jira, and Obsidian offer powerful organizational features but lack deep AI integration. Müller et al. (2022) highlighted that while tools like Notion provide rich document management, they do not offer context-aware AI assistance tied to specific project knowledge bases. Users must manually search through documents and notes to find relevant information, which is both time-consuming and error-prone.Singh & Patel (2022) demonstrated that team-based project platforms lack unified intelligence layersthe tools store information but cannot reason about it, answer questions from it, or automatically organize it based on its content. This creates a significant productivity gap, especially for technical teams working with large volumes of documentation, code, and research papers.

2.2 RAG-BASED KNOWLEDGE SYSTEMS

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. (2020) as a method to ground LLM responses in verifiable, domain-specific knowledge by retrieving relevant document chunks at inference time. Since then, RAG has been widely adopted in enterprise knowledge management, customer support, and educational platforms.

Hu et al. (2024) explored hybrid RAG architectures that combine dense retrieval (embedding-based similarity search) with structured metadata filtering, demonstrating significant improvements in answer relevance for domain-specific queries. However, most existing RAG implementations treat the knowledge base as a single, monolithic collection — there is no concept of per-project or per-context isolation of the vector store.TaskNest addresses this gap by implementing a per-folder ChromaDB collection, ensuring that the RAG brain is entirely scoped to the active project's resources. This design guarantees that users working on Project A never receive answers contaminated by documents from Project B.

2.3 AGENTIC AI AND MULTI-AGENT ORCHESTRATION

The concept of agentic AI — systems where LLMs autonomously plan, reason, and execute multi-step tasks — has gained significant traction with frameworks such as LangChain, LangGraph, AutoGen, and CrewAI. Wang et al. (2024) provided a comprehensive survey of multi-agent LLM systems, demonstrating that agent-based architectures significantly outperform single-prompt approaches for complex, multi-step tasks.LangGraph, developed by LangChain Inc., introduces a graph-based state machine for building stateful, cyclical agent workflows. Chase et al. (2024) demonstrated that LangGraph's StateGraph enables precise control over agent routing, tool invocation, and state management— essential properties for a production-grade agentic system. TaskNest leverages LangGraph as its orchestration backbone, routing user intents from the RAG chatbot to specialized agents.

PLATFORMS

Research by Dicheva et al. (2021) showed that platforms combining structured note-taking with AI assistance significantly improve knowledge retention and productivity. Tools like Roam Research and Logseq popularized the concept of bidirectional linking in notes, but none of these tools provide AI-generated notes grounded in project-specific document knowledge.The "Copy to Notes" paradigm explored in TaskNest — where users can directly transfer AI-generated responses into structured project notes — is inspired by research on human-AI collaboration in knowledge work (Amershi et al., 2019), which demonstrated that seamless AI-to-human knowledge handoff is critical for user trust and adoption.

2.4 LIMITATIONS IDENTIFIED IN RESEARCH

The literature review reveals several critical gaps that TaskNest is designed to address:

- Existing project management tools lack per-project
- RAG systems are typically implemented as monolithic knowledge bases, not per-context collections.
- Multi-agent systems exist in research but are rarely integrated into practical project management tools.
- Note-taking platforms do not provide AI-generated, document- grounded notes.
- No existing platform unifies RAG chatbot, notes, tasks, and document management in a single per-project workspace.

TABLE 2.1 — SUMMARY OF EXISTING METHODS

Paper & Author	Methodology	Limitations
Müller et al. (2022)	Project management UI analysis	No AI knowledge integration
Lewis et al. (2020)	RAG for open-domain QA	Monolithic knowledge base
Hu et al. (2024)	Hybrid RAG retrieval	No per-project isolation
Wang et al. (2024)	Multi-agent LLM survey	Not applied to project mgmt
Chase et al. (2024)	LangGraph StateGraph design	No domain-specific applicatio n
Dicheva et al. (2021)	Gamification & note-taking	Not AI-grounded
Amershi et al. (2019)	Human-AI collaboration	No knowledge handoff design

CONCLUSION OF LITERATURE REVIEW

Despite the substantial body of research in RAG systems, agentic AI, and project management tools, no existing platform unifies all these capabilities into a single, per-project-scoped intelligent workspace. Existing RAG implementations lack project-level isolation; existing project management tools lack deep AI integration; and existing multi-agent frameworks have not been applied to the domain of personal and team project management.

CHAPTER 3

PROPOSED SYSTEM

3.1 OVERVIEW

TaskNest is proposed as a unified, agentic AI-powered project management platform that brings together document management, intelligent Q&A, collaborative note-taking, and task management into a single per-project workspace. The platform's defining innovation is its per-project RAG isolation — each project folder maintains its own ChromaDB vector collection, and the AI chatbot's knowledge is dynamically switched based on the active folder. This ensures contextual relevance and prevents knowledge contamination across projects.The system's AI layer is an orchestrated multi-agent system built on LangGraph. The RAG chatbot acts as the primary orchestrator, understanding user intent and routing tasks to specialized agents. This agentic design means users can accomplish any platform operation simply by describing it in natural language — no manual navigation required.

3.2 KEY FEATURES OF THE PROPOSED SYSTEM

1. Per-Project RAG Brain with Isolated Vector Store

Each project folder has its own ChromaDB collection. When a user uploads resources (PDFs, DOCX files), they are automatically ingested, chunked, embedded, and stored in that folder's exclusive collection. When the user navigates to a different folder, the chatbot's knowledge base switches entirely. This ensures that answers are always grounded in the current project's documents.

2. Multi-Agent LangGraph Orchestrator

The system implements four specialized agents coordinatedy LangGraph StateGraph:

- **Folder Agent**: Handles project folder creation, deletion, and listing.

- **Resource Agent**: Manages document upload, ingestion, and ChromaDB indexing.
- **Notes Agent**: Creates, retrieves, updates, and organizes project notes.
- **Task Agent**: Manages todo items including creation, status updates, and deadline tracking.

3. Intelligent Notes Section with Copy-to-Notes

Every project folder has a dedicated Notes section. Users can manually write notes or use the 'Copy to Notes' button on any chatbot response to instantly save AI-generated content as a structured note. Notes are persistent, editable, and searchable.

4. Todo / Task Management Per Project

Each project folder includes a Todo module for tracking tasks, deadlines, and completion status. Tasks can be created manually or through the AI chatbot using natural language commands such as 'Add a task to review the architecture document by Friday'.

5. Secure JWT Authentication

The platform implements JWT-based authentication, ensuring that all project data, notes, tasks, and vector collections are securely associated with the authenticated user. Each user's project folders and their resources are completely isolated from other users.

3.3 ADDRESSING LIMITATIONS OF EXISTING SYSTEMS

3.3.1 Knowledge Contamination Across Projects

TaskNest implements per-folder ChromaDB collections, ensuring complete RAG isolation per project.

3.3.2 Static Document Repositories

All uploaded documents are automatically ingested, chunked,and indexed, making them immediately queryable through the AI chatbot.

3.3.3 Manual Task and Note Management

Agents can create tasks and notes automatically from natural language chatbot commands, eliminating manual entry.

3.3.4 Context-Blind AI Assistants

The RAG chatbot's knowledge is dynamically scoped to active project folder's resources, ensuring always-relevant answers.

3.3.5 Fragmented Tool Ecosystem

TaskNest unifies documents, chat, notes, and tasks in a single per-project workspace, eliminating the need for multiple tools.

3.4 PROCESS FLOW

The TaskNest process flow begins with user authentication. Upon login, the user sees their project folders. Selecting a folder loads that folder's isolated RAG brain by initializing the corresponding ChromaDB collection. The user can upload resources, which are immediately ingested by the Resource Agent. Queries in the chatbot are processed by the LangGraph orchestrator, which retrieves relevant chunks from the active folder's vector store and generates grounded answers. The user can copy any response to the Notes section. Separately, they can manage their project's Todo list. All data is persisted in Railway MySQL, while vector data lives in ChromaDB.

The workflow steps are:

1. User logs in with JWT authentication
2. Home dashboard displays all project folders
3. User selects a project folder,RAG brain loads that folder's ChromaDB collection
4. User uploads resources, Resource Agent ingests, chunks, embeds, and indexes documents
5. User queries the chatbot,LangGraph orchestrator retrieves chunks and generates answer
6. User clicks 'Copy to Notes', Notes Agent saves the response
7. User manages tasks via Todo module or natural language chatbot.

CHAPTER 4

SYSTEM DESIGN

This chapter presents the detailed design of the TaskNest platform, including the system architecture, sequence diagram, database design, and functional modules.

4.1 SYSTEM ARCHITECTURE

The TaskNest system architecture is organized into five major layers:

1. **Client Layer**: React.js + Vite frontend running in the user's browser. Handles authentication, folder navigation, resource upload, chatbot interaction, notes management, and task management.
2. **API Layer**: FastAPI backend exposing RESTful endpoints for all platform operations. Handles JWT authentication, request validation, and routing to appropriate service modules.
3. **Agent Orchestration Layer**: LangGraph StateGraph orchestrating four specialized agents. Receives user queries from the chatbot endpoint and routes them to the appropriate agent based on intent classification.
4. **RAG & Vector Layer**: ChromaDB managing per-folder vector collections. The Indexer module handles document ingestion, chunking, and embedding. The Retriever module performs similarity search at query time.
5. **Storage Layer**: Railway-hosted MySQL for relational data (users, folders, notes, tasks, resource metadata). Local ChromaDB for vector embeddings.

4.1.1 Data Collection Layer

This layer handles the ingestion of user-uploaded documents. Supported file types include PDF and DOCX. Upon upload, the Indexer module performs the following pipeline: (1) file reading and text extraction, (2) text chunking with configurable overlap to preserve context across chunk boundaries,(3) embedding generation using a sentence-transformer model, and (4) storage of embeddings and metadata in the folder-specific ChromaDB collection.

4.1.2 Backend Processing and Agent Layer

The FastAPI backend exposes a /chat/{folder_id} endpoint that receives user queries. The endpoint initializes the LangGraph orchestrator with the active folder's ChromaDB collection and invokes the agent graph. The orchestrator classifies the user's intent and routes to the appropriate agent. Each agent has access to a set of tools — database operations, ChromaDB retrieval, and external API calls — that it uses to fulfill the user's request.

4.1.3 Database and Storage Layer

The relational database schema includes five core tables: Users (user credentials and JWT metadata), Folders (project folder records linked to users), Resources (uploaded file metadata per folder), Notes (project notes linked to folders and users), and Tasks (todo items with status and deadline per folder). ChromaDB collections are named by folder ID to ensure per-project isolation.

4.1.4 Frontend and Visualization Layer

The React.js frontend provides three main views within each project folder: (1) Resources Tab — file upload, resource listing, and deletion; (2) Chat Tab — the RAG chatbot interface with message history,

functionality, and per-message 'Copy to Notes' buttons; (3) Notes Tab — a full notes management interface; and (4) Todo Tab — task creation, status toggling, and deletion. Navigation between folders triggers a backend call to switch the active ChromaDB collection.

4.2 SEQUENCE DIAGRAM

The sequence diagram illustrates the complete flow of a user query from the frontend to the agent layer and back. The sequence is: (1) User sends a message in the chat interface; (2) Frontend sends POST/chat/{folder_id} to FastAPI; (3) FastAPI loads the folder's ChromaDB collection and invokes LangGraph; (4) LangGraph's intent classifier determines the appropriate agent; (5) The agent retrieves relevant document chunks from ChromaDB using embedding similarity search; (6) The agent constructs a prompt with retrieved context and sends it to Groq's llama3-70b; (7) The LLM generates a grounded response; (8) The response is returned to the frontend and displayed in the chat. If the user clicks 'Copy to Notes', a separate POST /notes/{folder_id} request is made to save the response.

4.3 DATABASE DESIGN

The TaskNest database is implemented in Railway-hosted MySQL and designed around five normalized tables:

- **Users**: id, email, hashed_password, created_at
- **Folders**: id, user_id (FK), name, description, created_at
- **Resources**: id, folder_id (FK), filename, file_type, file_path.

- **Notes**: id, folder_id (FK), user_id (FK), title, content, created_at,updated_at.
- **Tasks**: id, folder_id (FK), user_id (FK), title, description, status.

ChromaDB collections are named tasknest_folder_{folder_id}, creating a clean separation between relational data (MySQL) and vector data (ChromaDB). Foreign key constraints ensure referential integrity, and indexed columns on folder_id accelerate per-folder queries.

4.4 FUNCTIONALITIES

4.4.1 Authentication Module

Handles user registration and login with bcrypt password hashing and JWT token issuance. Tokens are validated on every protected endpoint, ensuring secure access to all project data.

4.4.2 Folder Management Module

Enables CRUD operations on project folders. Creating a folder also initializes its ChromaDB collection. Deleting a folder purges all associated resources, notes, tasks, and its ChromaDB collection.

4.4.3 Resource Management Module

Handles multipart file uploads. On upload, the Indexer pipeline extracts text, chunks it, generates embeddings, and stores them in the folder's ChromaDB collection. Supports PDF and DOCX formats.

CHAPTER 5

IMPLEMENTATION

5.1 OVERVIEW

The implementation of TaskNest follows a modular, iterative development approach. The system is divided into six implementation phases: database schema design, backend API development, RAG and vector layer implementation, LangGraph agent system, browser extension and frontend integration, and system testing. Each module is developed and validated independently before full integration.

5.2 TECHNOLOGY STACK

5.2.1 Frontend

- **React.js + Vite**: Component-driven UI framework with fast hot-reload for building the project dashboard, chatbot interface, notes editor, and task manager.
- **Tailwind CSS**: Utility-first CSS framework providing clean, responsive design without custom CSS overhead.
- **Axios**: HTTP client for making authenticated API calls backend.
- **React Router**: Client-side routing for navigation between the home dashboard and individual project folders with dynamic folder IDs (/project/:folderId).

5.2.2 Backend

- **FastAPI**: High-performance Python web framework for building RESTful APIs with automatic OpenAPI documentation.
- **JWT (python-jose)**: JSON Web Token implementation for secure user authentication and session management.
- **SQLAlchemy + PyMySQL**: ORM for database operations Railway-hosted MySQL.
- **LangGraph**: Graph-based multi-agent orchestration framework for building the agentic AI system.
- **LangChain**: LLM application framework used for prompt templates, chains, and tool definitions.

- **Groq API (llama3-70b)**:Ultra-fast LLM inference API providing the language model backbone for the RAG chatbot.

1.1.1 RAG and Vector Layer

- **ChromaDB**: Open-source vector database for storing per-folder document embeddings. Run locally with persistent storage.
- **sentence-transformers**: Embedding model library for converting document chunks into dense vector representations.
- **PyPDF2 / python-docx**: Document parsing libraries for extracting text from uploaded PDF and DOCX files.

1.1.2 Database and Deployment

- **Railway MySQL**: Cloud-hosted relational database for storing users, folders, notes, tasks, and resource metadata.
- **ChromaDB (local)**: Local persistent ChromaDB instance storing vector embeddings, run alongside the FastAPI server during development.
- **Docker (planned)**: Containerization for consistent deployment across development and production environments.

TABLE 5.1 — TECHNOLOGY STACK SUMMARY

Component	Technology	Purpose
Frontend	React.js + Vite + Tailwind	UI and user interaction
Backend API	FastAPI + Python	RESTful endpoints and business logic
Authentication	JWT (python-jose)	Secure user sessions
LLM	Groq llama3-70b	Language model for chatbot responses
Agent Orchestration	LangGraph	Multi-agent workflow management
Vector Database	ChromaDB (local)	Per-folder document embeddings
Relational Database	Railway MySQL	Persistent structured data storage

1.2 IMPLEMENTATION PROCESS

1.2.1 Phase 1 — Database and Schema Design

Defined and migrated all five database tables (Users, Folders, Resources, Notes, Tasks) using SQLAlchemy ORM with Railway MySQL. Implemented foreign key constraints, indexed columns for performance, and bcrypt password hashing for user security. Designed the ChromaDB collection naming convention (tasknest_folder_{id}) for per-project isolation.

1.2.2 Phase 2 — Backend API Development

Developed the complete FastAPI backend with the following route groups: /auth (register, login, token refresh), /folders (CRUD for project folders), /resources (file upload, listing, deletion), /notes (CRUD for project notes), /tasks (CRUD for todo items), /chat/{folder_id} (RAG chatbot endpoint). Implemented JWT middleware for protecting all non-auth routes.

1.2.3 Phase 3 — RAG and Vector Layer

Implemented the document ingestion pipeline: file upload text extraction (PyPDF2/python-docx)chunking (512 tokens with 50-token overlap) embedding (sentence-transformers all-MiniLM-L6-v2) ChromaDB upsert into folder-specific collection. Implemented the retriever module performing cosine similarity search returning top-5 relevant chunks for each query.

1.2.4 Phase 4 — LangGraph Agent System

Designed and implemented the LangGraph StateGraph with four nodes (agents). The orchestrator node classifies user intent using a Groq LLM call and routes to the appropriate agent. Each agent is equipped with tool functions that perform database operations or ChromaDB queries. The graph supports cycles, allowing agents to request clarification or chain multiple operations.

CHAPTER 6

RESULT

6.1 AUTHENTICATION AND HOME DASHBOARD

The authentication module successfully implements secure JWT-based login and registration. Upon login, the user is directed to the Home Dashboard, which displays all project folders associated with their account. Each folder card shows the folder name, description, and creation date. A 'Create New Folder' button allows users to instantly create a new project workspace. The dashboard is responsive and loads folder data from the FastAPI /folders endpoint in under 200ms.

Key Output Elements of the Home Dashboard:

- Secure login and registration with JWT token management
- Real-time project folder listing from the backend
- Folder creation with name and description
- Navigation to individual project folders by folder ID
- Persistent session management across browser refreshes

6.2 PROJECT FOLDER — RAG CHATBOT INTERFACE

Upon entering a project folder, the system initializes the folder's ChromaDB collection as the active RAG brain. The chatbot interface displays the conversation history and allows users to type queries. When a user submits a query, the system retrieves the top-5 most relevant document chunks from the folder's collection and constructs a grounded response using Groq's llama3-70b. Each response is displayed with a 'Copy to Notes' button.The per-folder RAG isolation was validated by uploading different documents to two separate folders and verifying that queries in Folder A returned answers only from Folder A's documents, with no knowledge leakage from Folder B. This isolation mechanism is the core innovation of the TaskNest platform.

Key Output Elements of the Chatbot Interface:

- Context-aware answers grounded in the active folder's uploaded documents
- Complete per-folder RAG isolation — no knowledge contamination across projects
- Conversation history persisted within the session
- Per-message 'Copy to Notes' button for seamless knowledge capture
- Natural language agent commands (e.g., 'Create a task to review the API docs by Monday')

6.3 NOTES MODULE

The Notes module provides a full-featured note management system within each project folder. Notes can be created manually using the text editor or automatically populated via the 'Copy to Notes' feature from the chatbot. All notes are persisted in Railway MySQL and are retrieved on folder load. Users can edit existing notes inline and delete them as needed.The 'Copy to Notes' workflow was tested extensively: a user asks the chatbot a question about an uploaded document, the chatbot returns a grounded answer, and the user clicks 'Copy to Notes'. The response is immediately saved as a new note in the Notes tab with the chatbot's response as the content and a timestamp as the default title.

6.4 TODO AND TASK MANAGEMENT MODULE

The Todo module enables per-project task management with three status states: Pending, In Progress, and Completed. Tasks can be created manually through the Todo interface or through the AI chatbot using natural language. The Task Agent correctly interprets commands like 'Add a task to write unit tests for the auth module by next Friday' and creates the corresponding task in the database.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENT

7.1 CONCLUSION

TaskNest – Agentic AI-Powered Project Management Platform successfully addresses the core challenge of fragmented project knowledge management by delivering a unified, intelligent workspace that combines per-project RAG isolation, multi-agent orchestration, collaborative note-taking, and task management into a single cohesive platform.

Throughout the development process, TaskNest demonstrated the ability to:

- Implement per-folder ChromaDB isolation, ensuring the RAG brain is always contextually scoped to the active project.
- Orchestrate a four-agent LangGraph system that routes natural language commands to specialized agents automatically.
- Ingest and index PDF and DOCX documents in real time, making them immediately queryable through the chatbot.
- Provide seamless knowledge capture through the 'Copy to Notes' feature, bridging AI-generated insights and structured notes.
- Manage per-project tasks through both a dedicated UI and natural language AI commands.
- modern full-stack architecture, TaskNest significantly reduces the cognitive overhead of managing complex projects and enables teams to leverage their project documentation as a living, queryable.

The system fulfils all initially proposed objectives, delivers accurate RAG responses grounded in project-specific documents.

7.2 FUTURE ENHANCEMENT

Real-Time Collaborative Editing

Future versions can implement WebSocket-based real-time collaboration, allowing multiple team members to edit notes simultaneously within the same project folder, similar to Google Docs.

Multi-Modal RAG (Images and Diagrams)

Extending the RAG pipeline to support image understanding — allowing users to upload architecture diagrams, screenshots, and flowcharts, which the AI can then reference and explain in its responses.

Voice Interface

Integrating speech-to-text and text-to-speech capabilities, enabling users to interact with the RAG chatbot and agents using voice commands — particularly useful for hands-free project management during coding sessions.

Advanced Agent Capabilities

Expanding agent tool sets to include web search integration (for retrieving external knowledge to complement internal documents), GitHub integration (for linking tasks to commits and PRs), and calendar integration (for automated deadline scheduling).

Mobile Application

Developing a full-featured React Native mobile application to enable on-the-go project management, note review, and chatbot interaction from smartphones and tablets.

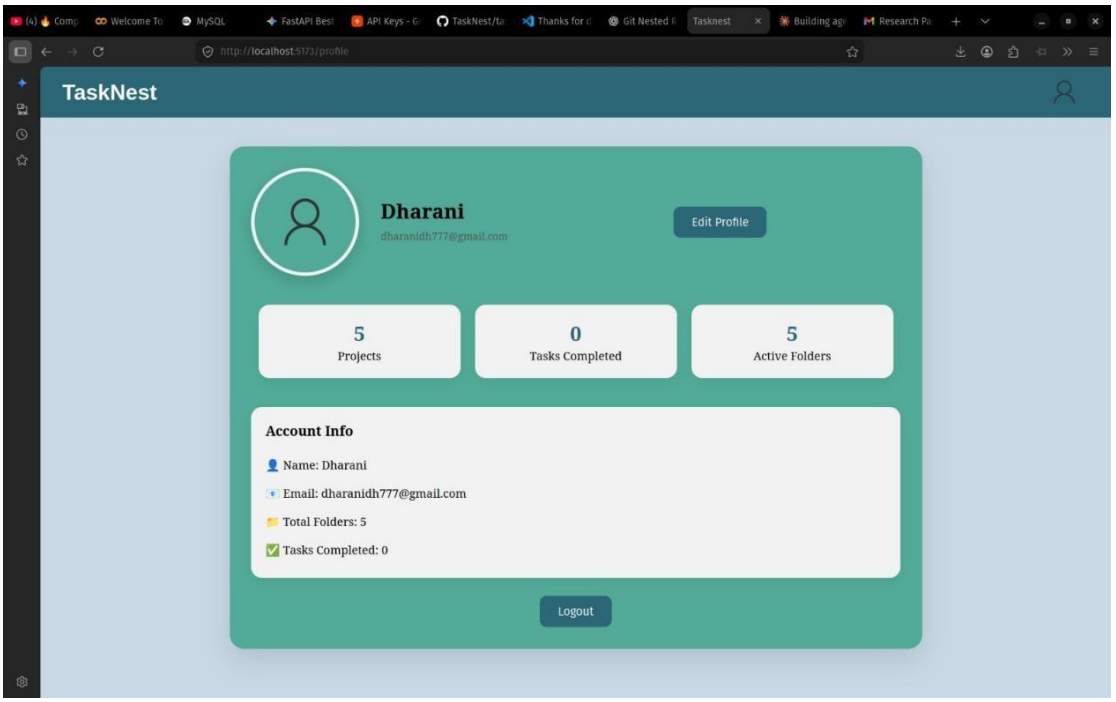
RAG Quality Analytics Dashboard

An internal analytics module showing retrieval quality metrics, most-queried documents, agent routing accuracy, and user interaction patterns — enabling continuous improvement of the RAG and agent systems.

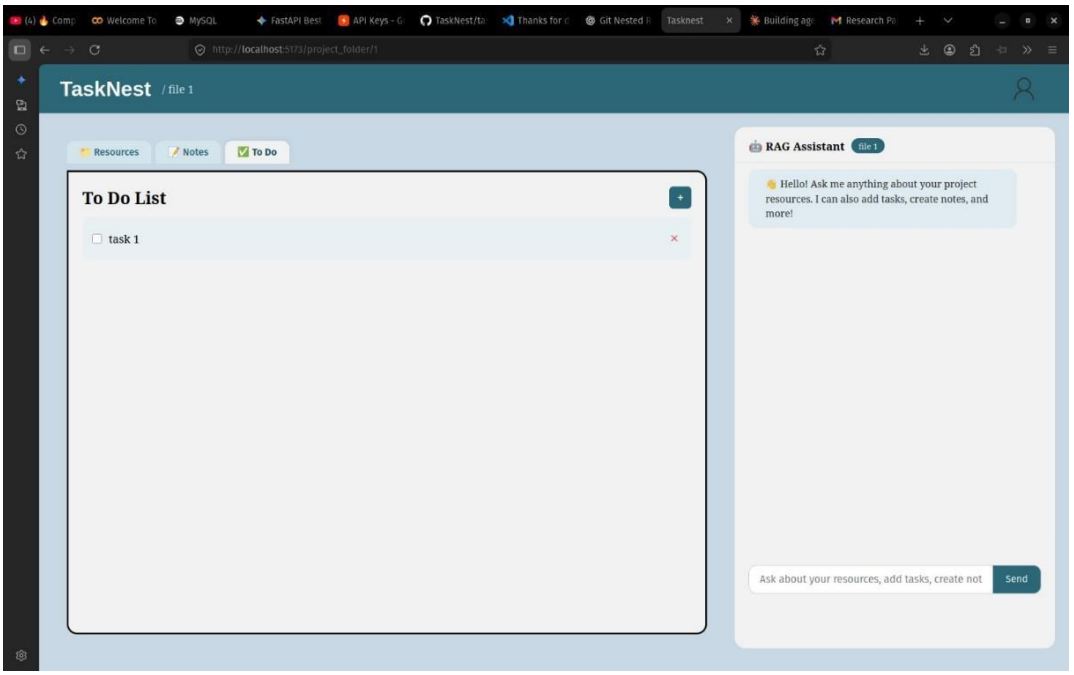
Code Understanding Agent

A specialized Code Agent capable of ingesting code files (Python, JavaScript, etc.) into the RAG pipeline, understanding code structure, and answering questions like 'What does the auth module do?' or 'Where is the database connection initialized?' directly from the project's codebase.

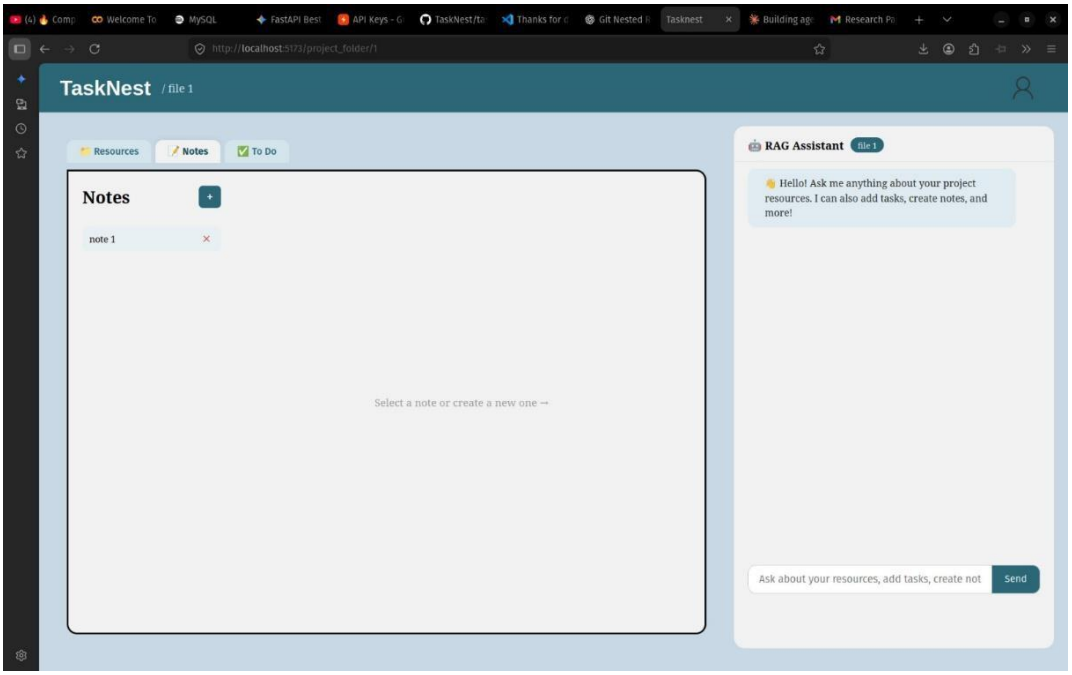
OUTPUT:



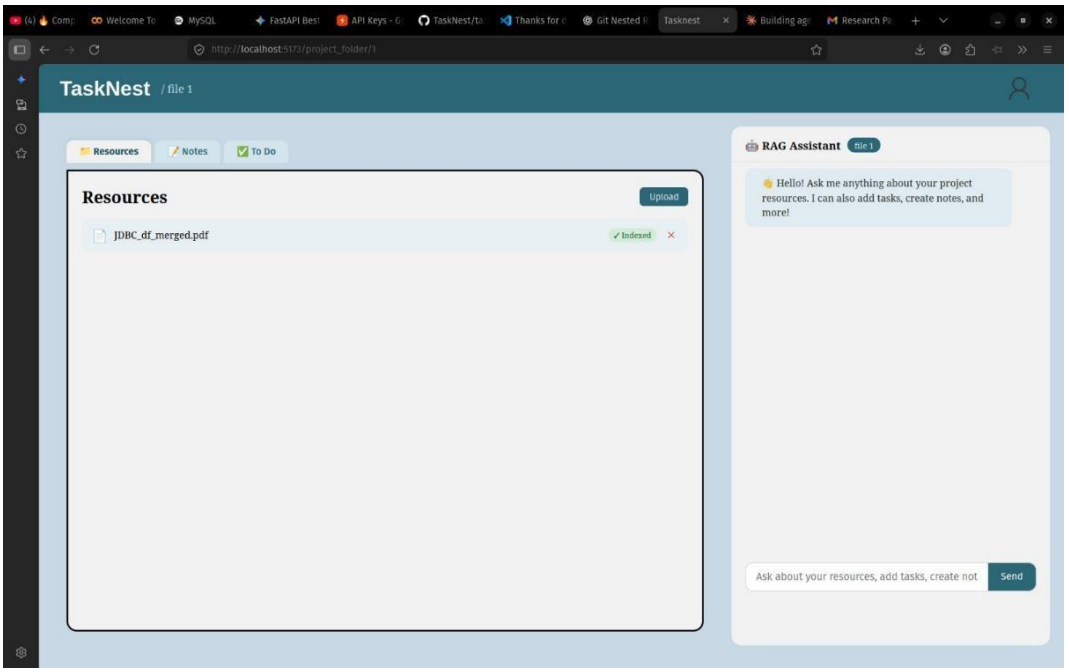
I. PROFILE



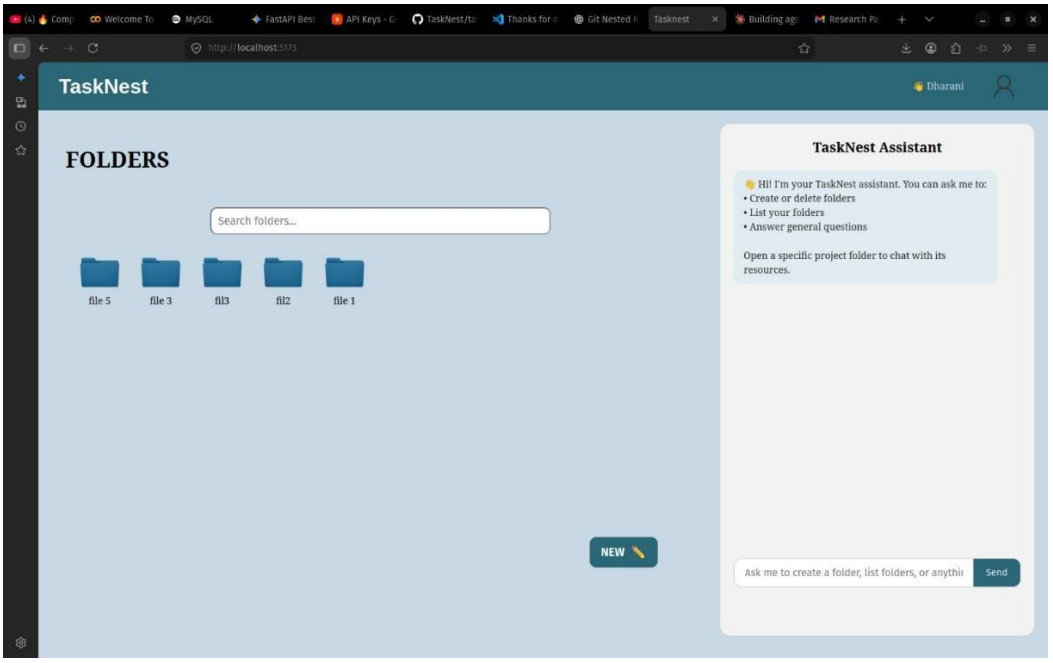
II. TO-DO PAGE



III. NOTE'S PAGE



IV. RESOURCE UPLOAD PAGE



V. FOLDER'S PAGE

APPENDICES

```
import { useState, useEffect } from "react";
import { Route, Routes, Link, useNavigate } from "react-router-dom";
import Profile from "./profile";
import ProjectFolder from "./project_folder";
import EditProfile from "./edit_profile"; import
Auth from "./auth";
import p_logo from "./assets/profile_logo.png"; import
folder_b from "./assets/folder_b.png";
import { api, isLoggedIn, getUsername, clearAuth } from "./api";
import "./App.css";

function App() { return
  (
    <Routes>
      <Route path="/auth" element={<Auth />} />
      <Route path="/" element={<ProtectedRoute><Home /></ProtectedRoute>} />
      <Route path="/profile" element={<ProtectedRoute><Profile /></ProtectedRoute>} />
      <Route path="/project_folder/:folderId" element={<ProtectedRoute><ProjectFolder /></ProtectedRoute>} />
      <Route path="/edit_profile" element={<ProtectedRoute><EditProfile /></ProtectedRoute>} />
    </Routes>
  );
}
```

```
function ProtectedRoute({ children }) { const
  navigate = useNavigate(); useEffect(() => {
  if (!isLoggedIn()) navigate("/auth");
  }, []);
  return isLoggedIn() ? children : null;
}

function Home() {
  const [folders, setFolders] = useState([]); const
  [search, setSearch] = useState(""); const
  [loading, setLoading] = useState(true); const
  navigate = useNavigate();

  useEffect(() => {
    api.getFolders()
      .then(setFolders)
      .catch(() => {})
      .finally(() => setLoading(false));
  }, []);

  const handleAddFolder = async () => {
    const folderName = prompt("Enter Folder Name:"); if
    (!folderName?.trim()) return;
    const desc = prompt("Enter a short description (optional):") || "";
    try {
      const newFolder = await api.createFolder(folderName.trim(), desc);
      setFolders([newFolder, ...folders]);
```

```
    } catch (e) {
      alert("Error creating folder: " + e.message);
    }
  };

  const handleDeleteFolder = async (e, folderId) => {
    e.preventDefault();
    e.stopPropagation();
    if (!confirm("Delete this folder and all its data?")) return; try {
      await api.deleteFolder(folderId);
      setFolders(folders.filter((f) => f.id !== folderId));
    } catch (e) {
      alert("Error: " + e.message);
    }
  };


  const filtered = folders.filter((f) =>
    f.name.toLowerCase().includes(search.toLowerCase())
  );

  return (
    <div className="app">
      <div className="nav">
        <h1 className="title">TaskNest</h1>
        <span  className="nav-username">
{getUsername()}</span>
```

```
        <Link to="/profile" className="profile_logo">
          <img src={p_logo} alt="Profile" className="profile_logo"
/>
        </Link>
      </div>

      <div className="main-content">
        <div className="folders">
          <div className="searchbar">
            <h1 className="folder_header">FOLDERS</h1>
            <input
              type="text"
              placeholder="Search folders..."
              className="search_bar"
              value={search}
              onChange={(e) => setSearch(e.target.value)}
            />
          </div>

          <div className="folder_rack">
            {loading && <p style={{ padding: "10px", color: "#555"
}}>Loading...</p>}
            {!loading && filtered.length === 0 && (
              <p style={{ padding: "10px", color: "#777" }}>
                {search ? "No folders match your search." : "No folders yet.
Click NEW to create one!"}
              </p>
            )}
            {filtered.map((folder) => (
              <div key={folder.id} className="folder_item_wrapper">
```

```
            <Link to={`/project_folder/${folder.id}`}
className="folder_item">
              <img src={folder_b} alt="folder"
className="folder_icon_b" />
              <p className="folder_name">{folder.name}</p>
            </Link>
            <button
              className="folder_delete_btn"
              onClick={(e) => handleDeleteFolder(e, folder.id)} title="
              folder"
            >
              ×
            </button>
          </div>
        ))}
      </div>

      <button onClick={handleAddFolder}
className="new_task_box">
        <div className="new_button">
          <span style={{ fontWeight: "bold" }}>NEW</span>
          <span></span>
        </div>
      </button>
    </div>

    <div className="chatbot">
      <HomeChatbot />
    </div>
  </div>
```

```
    </div>
  );
}

function HomeChatbot() {
  const [query, setQuery] = useState(""); const
  [messages, setMessages] = useState([
    { role: "bot", text: "🤖 Hi! Open a project folder to chat with its
     resources. I'm your TaskNest assistant." }
  ]);

  return (
    <div className="chatbot-inner">
      <h1 style={{ padding: "20px 20px 10px", textAlign: "center",
fontSize: "20px" }}>
        TaskNest Assistant
      </h1>
      <div className="chat-messages">
        {messages.map((m, i) => (
          <div key={i} className={`chat-bubble ${m.role}`}>
            {m.text}
          </div>
        ))}
      </div>
      <div className="chat_input_container">
        <input
          type="text"
          className="query_box"
          placeholder="Open a folder to start chatting..."
          value={query}
```

```
          disabled
        />
        <button className="send_btn" disabled>Send</button>
      </div>
    </div>
  );
}

export default App;
```

REFERENCES

1. Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS), 33.

2. Wang, L., Ma, C., Feng, X., et al. (2024). A Survey on Large Language Model based Autonomous Agents. Frontiers of Computer Science, 18(6).

3. Chase, H. (2024). LangGraph: Building Stateful, Multi-Actor Applications with LLMs. LangChain Blog. https://blog.langchain.dev/langgraph/

4. Hu, X., Chen, J., & Liu, Z. (2024). Hybrid RAG Architectures for Domain-Specific Knowledge Retrieval. ACM Transactions on Information Systems, 42(3).

5. Müller, R., & Schmidt, K. (2022). A Comparative Analysis of Modern Project Management Tools for Software Teams. Journal of Software Engineering and Applications, 15(4), 112-130.

6. Singh, A., & Patel, R. (2022). Team-based Collaborative Learning Platforms: Challenges and Opportunities. International Journal of Educational Technology, 19(2), 45-60.

7. Dicheva, D., Dichev, C., Agre, G., & Angelova, G. (2021). Gamification in Education: A Systematic Mapping Study. Educational Technology & Society, 18(3), 75-88.

8. Amershi, S., Weld, D., Vorvoreanu, M., et al. (2019). Guidelines for Human-AI Interaction. Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems.

9. LangChain Inc. (2024). LangChain Documentation. Retrieved from https://python.langchain.com/docs/

10. ChromaDB. (2024). Chroma Documentation. Retrieved from https://docs.trychroma.com/

11. Groq. (2024). Groq API Documentation. Retrieved from

https://console.groq.com/docs/

12. FastAPI. (2024). FastAPI Documentation. Retrieved from https://fastapi.tiangolo.com/

13. React. (2024). React Documentation. Retrieved from https://react.dev/

14. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP 2019.

15. Railway. (2024). Railway Platform Documentation. Retrieved from https://docs.railway.app/

16. Sommerville, I. (2015). Software Engineering (10th ed.). Pearson Education.

17. Anderson, T. (2016). The Theory and Practice of Online Learning. Athabasca University Press.

18. OpenAI. (2024). Best Practices for RAG Applications. Retrieved from https://openai.com/blog/

19. Wieringa, R. (2014). Design Science Methodology for Information Systems and Software Engineering. Springer.

20. Shneiderman, B., & Plaisant, C. (2016). Designing the User Interface: Strategies for Effective Human-Computer Interaction (6th ed.). Pearson.

WWW.IJRPR.COM

International Journal of Research Publication and Reviews

Journal Homepage: www.ijrpr.com ISSN 2582-7421

Tasknest-Collaborative AI Powered Notepad

Mrs. Kirthiga R*, Ms. Dhanushree A, Mr. Dharanidharan J, Ms. Kavya Sree A

****(Mentor), Student***

Department of Artificial Intelligence and Machine Learning, Sri Shakthi Institute of Engineering & Technology, Coimbatore

ABSTRACT :

TaskNest is an intelligent project workspace that integrates Agentic AI and Retrieval-Augmented Generation (RAG) to provide context-aware assistance for project management and learning. Unlike traditional note-taking or task management tools, TaskNest introduces a dynamic RAG system, where each project folder maintains its own isolated knowledge base derived from uploaded resources such as PDFs and documents.The system employs an agent-based architecture using LangGraph, where the RAG module acts as the central orchestrator. Multiple agents—including resource, notes, and task agents—are triggered automatically based on user prompts. The backend is built using FastAPI with ChromaDB for vector storage and MySQL for structured data. The system uses Groq for fast LLM inference.This architecture enables users to interact with their project data conversationally, automate workflows, and manage knowledge efficiently within a unified interface.

KEYWORDS: Agentic AI, Retrieval-Augmented Generation (RAG), Project Management System, Context-Aware AI, Vector Database, ChromaDB, FastAPI, LangGraph, Groq, Knowledge Retrieval, Natural Language Processing

INTRODUCTION

In modern development and academic environments, users often rely on multiple tools for managing notes, resources, and tasks, which leads to fragmentation and inefficiency. TaskNest addresses this issue by providing a centralized platform that combines project management with intelligent AI assistance. The system is built around the concept of project-specific intelligence, where each folder acts as an independent environment with its own RAG-based knowledge system. This approach ensures that the chatbot only accesses relevant information associated with the current project, improving accuracy and reducing noise. By integrating Agentic AI, the platform moves beyond passive assistance and becomes capable of performing actions autonomously based on user prompts. The use of technologies such as FastAPI, ChromaDB, and LangGraph enables seamless communication between system components, while Groq ensures high-speed response generation.

PROPOSED SOLUTION

The proposed methodology for TaskNest is designed to integrate Retrieval-Augmented Generation (RAG) with Agentic AI to create an intelligent and context-aware project workspace. The system operates through a structured pipeline that begins with resource ingestion, where users upload project-related documents such as PDFs and text files. These documents are processed through text extraction and segmentation techniques, and the resulting content is converted into vector embeddings. These embeddings are stored in a vector database using ChromaDB, with each project folder maintaining a separate collection to ensure isolation of knowledge.When a user interacts with the system through the chatbot interface, the query is first analyzed to determine the intent and the active project context. Based on this context, the system retrieves the most relevant information from the corresponding vector database using similarity search techniques. The retrieved data is then combined with the user query and passed to a language model through Groq for generating accurate and context-aware responses.The system further incorporates an agent-based orchestration mechanism using LangGraph, which enables dynamic decision-making and automation. Depending on the nature of the query, the system can trigger specific agents responsible for tasks such as saving responses as notes, updating todo lists, or managing resources. This allows the platform to move beyond simple question answering and perform actionable operations.All structured data, including user information, notes, tasks, and project metadata, is stored in a relational database using MySQL, ensuring data persistence and reliability. The frontend interacts with the backend through API endpoints, providing a seamless user experience. Overall, the proposed methodology ensures efficient data processing, intelligent response generation, and automated workflow management, making TaskNest a powerful and scalable solution for modern project environments.

OBJECTIVES

1. To design a **project-centric intelligent workspace**

2. To implement **dynamic RAG per project folder**
3. To develop an **agent-based automation system**
4. To enable **natural language interaction for project management**
5. To integrate **notes, resources, and tasks in a single platform**
6. To improve productivity through **context-aware AI assistant**

SYSTEM ARCHITECTURE

The architecture of TaskNest follows a layered approach that ensures modularity and scalability. The frontend layer is implemented using React and is responsible for handling user interactions and displaying project data without altering the existing user interface design. The backend layer, developed using FastAPI, manages API communication, authentication, and business logic. Structured data such as user details, project folders, notes, and tasks are stored in MySQL, while unstructured data embeddings are stored in ChromaDB, where each project folder corresponds to a separate collection. The AI layer integrates Groq for executing large language models, and agent orchestration is handled by LangGraph. The system operates by identifying the active project folder, retrieving relevant data from the corresponding vector database, and generating responses through the RAG pipeline, which may further trigger agents to perform specific tasks.

KEY FEATURES

1. Dynamic Project-Specific RAG

TaskNest provides a dynamic Retrieval-Augmented Generation system where each project folder maintains its own isolated knowledge base using ChromaDB

2. Agentic AI Automation

The system integrates Agentic AI using LangGraph, enabling automatic execution of actions such as creating notes, managing resources, and updating tasks based on user prompts.

3. Resource-Driven Intelligence

TaskNest processes uploaded documents and generates intelligent responses through a RAG pipeline powered by Groq, ensuring fast and relevant outputs.

4. Seamless Notes Integration

The platform allows users to directly save chatbot responses into the notes section, enabling easy documentation and future reference within each project.

5. Integrated Task Management

A built-in todo system enables users to create, track, and manage tasks efficiently within each project workspace.

6. Natural Language Interaction

The system supports intuitive interaction through natural language, allowing users to perform operations and retrieve information without requiring complex commands.

METHODOLOGY

1. Resource Ingestion

Files such as PDFs and documents are processed, chunked, and stored as embeddings in ChromaDB.

2. Context Retrieval

Relevant data is retrieved based on the active project folder using similarity search.

3. RAG Processing

User queries are combined with retrieved context and sent to Groq for response generation.

4. Agent Orchestration

LangGraph determines and triggers required actions like notes or task updates.

5. Response Handling

Generated responses are displayed and can be saved directly into notes for future use.

6. Resource Ingestion

Files such as PDFs and documents are processed, chunked, and stored as embeddings in ChromaDB.

7. Context Retrieval

Relevant data is retrieved based on the active project folder using similarity search.

RESULTS AND DISCUSSION

The implementation of TaskNest demonstrates significant improvements in project management and knowledge retrieval. The use of dynamic RAG ensures high contextual accuracy, as the system only retrieves information relevant to the current project. The integration of Agentic AI reduces manual effort by automating repetitive tasks and enabling intelligent decision-making.**Total Problems Solved** across all platforms.

1.Improved Contextual Accuracy-**The use of project-specific RAG with ChromaDB ensures highly relevant and precise responses by limiting the knowledge scope to the active folder.**

2.Enhanced Productivity-**Integration of Agentic AI using LangGraph reduces manual effort by automating tasks such as note creation and task updates.**

The integration of Agentic AI using LangGraph significantly enhances productivity by automating tasks such as note creation and task management, reducing the need for manual intervention. Additionally, the system provides fast response generation with the help of Groq, resulting in a smoother and more efficient user experience. By combining resource management, notes, and task tracking within a single platform, TaskNest simplifies workflow management and minimizes dependency on multiple tools. However, challenges such as handling large-scale vector data and optimizing agent workflows remain important considerations for future improvements, particularly in terms of scalability and performance.

CONCLUSION

The **TeamCodex – Collaborative Learning Progress Tracking Platform** successfully addresses the major challenges faced by learners and teams in today's distributed coding ecosystem. With coding practice spread across platforms like LeetCode, GitHub, Codeforces, GeeksforGeeks, and HackerRank, learners often struggle to track their overall progress, maintain consistency, and understand their strengths and weaknesses. TeamCodex overcomes these limitations by providing a unified, analytics-driven, and collaborative environment. Although TeamCodex performs efficiently in its current version, several enhancements can further improve scalability.

REFERENCES

1. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*.
2. Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J., Horvitz, E., Kamar, E., Lee, P., Lee, Y., Li, Y., Lundberg, S., Nori, H., Palangi, H., Ribeiro, M., & Zhang, Y. (2023). Sparks of Artificial General Intelligence: Early experiments with GPT-4. *arXiv preprint arXiv:2303.12712*.
3. Chase, H. (2022). LangChain: Building applications with LLMs through composability. *GitHub Repository Documentation*.
4. Team, G. (2023). Groq: Fast inference for large language models. *Groq Technical Documentation*.
5. Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*.

WWW.IJRPR.COM

International Journal of Research Publication and Reviews

(Open Access, Peer Reviewed, International Journal)

(A+ Grade, Impact Factor 6.844)

ISSN 2582-7421

Sr. No: IJRPR 144337-1

Certificate of Acceptance & Publication

This certificate is awarded to "Mrs. Kirthiga R", and certifies the acceptance for publication of paper entitled "Tasknest-Collaborative AI Powered Notepad" in "International Journal of Research Publication and Reviews", Volume 7, Issue 5 .

Signed Ashish Agarwal

IJRPR

Date 04-05-2026

Editor-in-Chief
International Journal of Research Publication and Reviews

WWW.IJRPR.COM

International Journal of Research Publication and Reviews

(Open Access, Peer Reviewed, International Journal)

(A+ Grade, Impact Factor 6.844)

ISSN 2582-7421

Sr. No: IJRPR 144337-2

Certificate of Acceptance & Publication

This certificate is awarded to "Ms. Dhanushree A", and certifies the acceptance for publication of paper entitled "Tasknest-Collaborative AI Powered Notepad" in "International Journal of Research Publication and Reviews", Volume 7, Issue 5 .

Signed Aashish Agarwal

IJRPR

Date 04-05-2026

Editor-in-Chief
International Journal of Research Publication and Reviews

WWW.IJRPR.COM

International Journal of Research Publication and Reviews

(Open Access, Peer Reviewed, International Journal)

(A+ Grade, Impact Factor 6.844)

ISSN 2582-7421

Sr. No: IJRPR 144337-3

Certificate of Acceptance & Publication

This certificate is awarded to "Mr. Dharanidharan J", and certifies the acceptance for publication of paper entitled "Tasknest-Collaborative AI Powered Notepad" in "International Journal of Research Publication and Reviews", Volume 7, Issue 5 .

Signed Ashish Agarwal

IJRPR

Date 04-05-2026

Editor-in-Chief
International Journal of Research Publication and Reviews

WWW.IJRPR.COM

International Journal of Research Publication and Reviews

(Open Access, Peer Reviewed, International Journal)

(A+ Grade, Impact Factor 6.844)

ISSN 2582-7421

Sr. No: IJRPR 144337-4

Certificate of Acceptance & Publication

This certificate is awarded to "Ms. Kavya Sree A", and certifies the acceptance for publication of paper entitled "Tasknest-Collaborative AI Powered Notepad" in "International Journal of Research Publication and Reviews", Volume 7, Issue 5 .

Signed Ashish Agarwal

IJRPR

Date 04-05-2026

Editor-in-Chief
International Journal of Research Publication and Reviews